Tecnológico de Antioquia

Aplicación de transferencias bancarias

Informe del ejercicio propuesto — Laboratorio Spring Boot con Cursor

---

Curso: Proyecto de Aula

Estudiante: Omar Tamara

Repositorio: https://github.com/T4mDev/BancoTransferencias-TdeA

Contenido

1. Objetivo
2. Stack tecnológico
3. Arquitectura
4. Modelo de datos
5. Reglas de negocio
6. Endpoints de la API
7. Generación del código QR
8. Frontend
9. Pruebas y evidencias
10. Prompts utilizados
11. Ejecución
12. Conclusiones

1. Objetivo

Desarrollar, siguiendo los pasos del laboratorio, una aplicación completa para trasladar dinero entre cuentas de un banco. La aplicación debe mostrar el histórico de transacciones y, al generar el resultado de cada transacción, presentar un código QR con sus datos básicos codificados. El desarrollo se apoyó en el IDE Cursor y en prompts para generar cada capa del sistema.

2. Stack tecnológico

Capa	Tecnología
Backend	Java 21+ (compatible con Java 25), Spring Boot 4.0.9, Spring Web MVC, Spring Data JPA / Hibernate, Bean Validation
Base de datos	PostgreSQL (base banktdea). H2 en memoria para las pruebas automáticas
Código QR	ZXing 3.5.4 (core + javase)
Documentación API	springdoc-openapi 3 con Swagger UI
Frontend	React 19, Vite 6, Axios
Herramientas	Cursor con reglas .mdc, Maven, Node.js, Git y GitHub, pgAdmin

3. Arquitectura

El backend usa una arquitectura en capas y el frontend es una aplicación de una sola página (SPA) que consume la API REST:

- **Controller:** recibe las peticiones HTTP, valida los datos con @Valid y define el código de respuesta.
- **Service:** contiene las reglas de negocio, maneja las transacciones de base de datos y genera el código QR.
- **Repository:** acceso a datos con Spring Data JPA, incluido el bloqueo de filas para las transferencias.
- **Entity / DTO:** las entidades JPA representan las tablas; los DTO son records de Java y son lo único que sale de la API.
- **Frontend:** React consume la API con Axios; la URL del backend viene de la variable de entorno VITE_API_URL. El backend tiene un filtro CORS que acepta peticiones de http://localhost:3000.

```
React (localhost:3000) --Axios--> Controller --> Service --> Repository --> PostgreSQL
```

Estructura del proyecto

```
BancoTransferencias-TdeA/
├── .cursor/rules/springboot.mdc
├── backend/src/main/java/com/tdea/banktransfer/
│   ├── config/       CorsConfig, DataSeeder
│   ├── controller/   AccountController, TransferController
│   ├── dto/          Records de entrada y salida
│   ├── entity/       Account, Transaction, TransactionStatus
```

```
│   ├── exception/    BusinessException (sealed), GlobalExceptionHandler
│   ├── repository/   AccountRepository, TransactionRepository
│   └── service/      AccountService, TransferService, QrCodeService
├── frontend/src/     bankService.js, componentes React
└── docs/             Informe, PROMPTS.md, capturas
```

4. Modelo de datos

Tabla accounts

Campo	Tipo	Descripción
id	BIGINT (PK)	Identificador autogenerado
account_number	VARCHAR(20) único	Número de cuenta (6 a 20 dígitos)
owner_name	VARCHAR(120)	Titular
balance	NUMERIC(19,2)	Saldo actual
created_at	TIMESTAMP	Fecha de apertura
version	BIGINT	Control de concurrencia optimista

Tabla transactions (histórico)

Campo	Tipo	Descripción
id	BIGINT (PK)	Identificador autogenerado
reference	VARCHAR(36) único	Referencia pública (UUID)
source_account	VARCHAR(20)	Cuenta origen
target_account	VARCHAR(20)	Cuenta destino
amount	NUMERIC(19,2)	Monto transferido
description	VARCHAR(200)	Descripción opcional
status	VARCHAR(20)	COMPLETED o REJECTED
failure_reason	VARCHAR(255)	Motivo del rechazo
created_at	TIMESTAMP	Fecha de la transacción

5. Reglas de negocio

Una transferencia solo se completa si se cumplen todas estas condiciones; de lo contrario se lanza una excepción personalizada:

Regla	Excepción	HTTP
Las dos cuentas deben existir	AccountNotFoundException	404
Origen y destino deben ser distintas	InvalidTransferException	400
El monto debe ser mayor que cero	Validación / InvalidTransfer	400

Regla	Excepción	HTTP
La cuenta origen debe tener saldo suficiente	InsufficientFundsException	422
No se repiten números de cuenta al abrir una	DuplicateAccountException	409

Aspectos técnicos importantes:

- **Atomicidad:** el débito, el crédito y el registro de la transacción ocurren dentro de una sola transacción de base de datos; si algo falla, no se mueve dinero.
- **Concurrencia:** las dos cuentas se bloquean con PESSIMISTIC_WRITE y siempre en el mismo orden (por número de cuenta), lo que evita saldos inconsistentes y deadlocks entre transferencias cruzadas.
- **Histórico completo:** cuando una regla falla se hace rollback y el intento se guarda aparte con estado REJECTED y su motivo, para que también aparezca en el histórico.
- **Dinero:** todos los montos usan BigDecimal, nunca double, para evitar errores de redondeo.
- **Errores:** las excepciones forman una jerarquía sealed y se devuelven en formato ProblemDetail (RFC 9457) con mensajes en español.

6. Endpoints de la API

Método	Ruta	Descripción
GET	/api/cuentas	Listar cuentas
GET	/api/cuentas/{numero}	Consultar una cuenta
POST	/api/cuentas	Abrir una cuenta (201)
POST	/api/transferencias	Transferir; devuelve la transacción y el QR (201)
GET	/api/transacciones?cuenta=	Histórico, con filtro opcional por cuenta
GET	/api/transacciones/{ref}/qr	Imagen PNG del QR de una transacción

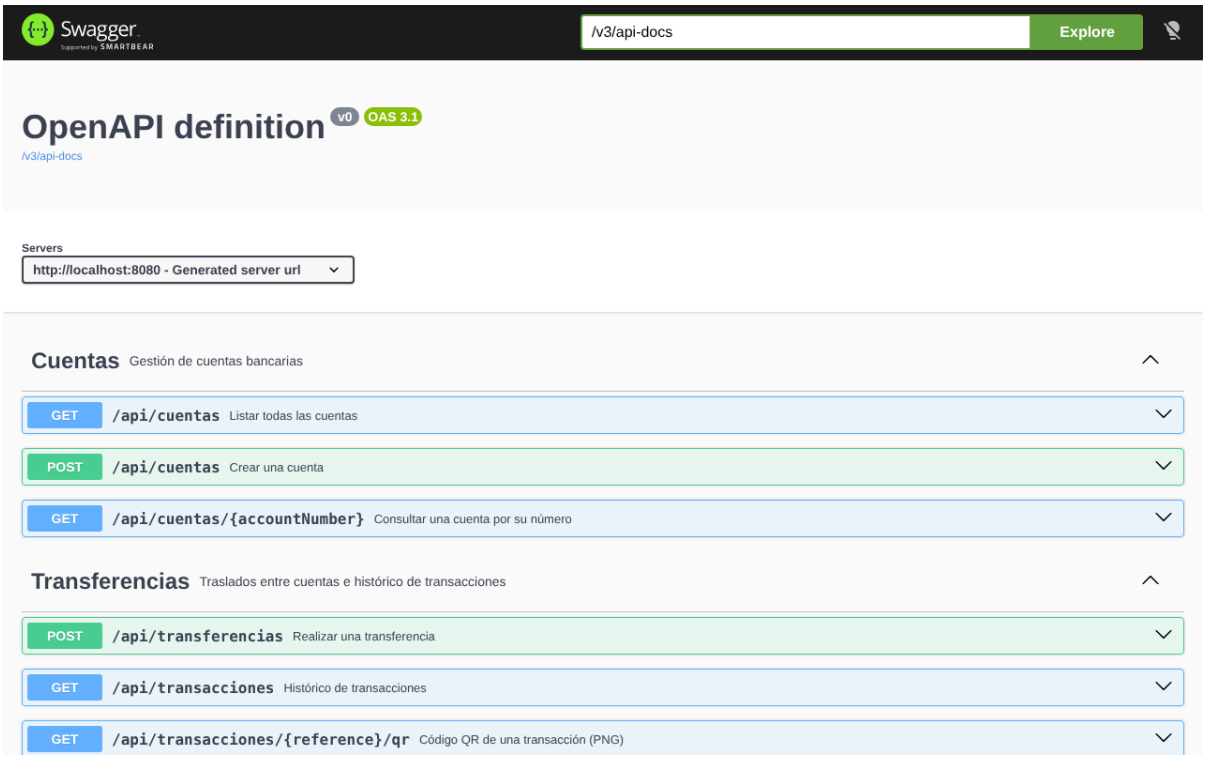


Figura 1. Documentación de la API en Swagger UI

7. Generación del código QR

La clase QrCodeService usa la librería ZXing para codificar en un QR de 300 × 300 píxeles (UTF-8, corrección de errores nivel M) el siguiente texto:

```
BANCO TdeA - Comprobante de transferencia
Referencia: 12b68f93-f37f-44c0-8caf-cf0ed2056306
Cuenta origen: 1001001001
Cuenta destino: 1001001002
Monto: $ 150.000,00
Fecha: 2026-10-31 11:55:13
Estado: COMPLETED
```

Al completar una transferencia, la API devuelve el QR como data URI en base64 para mostrarlo de inmediato en el comprobante. Para cualquier transacción del histórico, el endpoint /api/transacciones/{ref}/qr devuelve la imagen PNG. Se verificó leyendo el QR generado con un decodificador independiente (OpenCV), que devolvió exactamente el texto anterior.

8. Frontend

La interfaz en React tiene cuatro partes: el panel de cuentas (al hacer clic en una tarjeta se usa como cuenta origen y se pueden abrir cuentas nuevas), el formulario de transferencia (avisa si el monto supera el saldo disponible), el comprobante con el código QR descargable y la tabla del histórico con

filtro por cuenta. Los errores del backend se muestran en un aviso tipo toast. El diseño se adapta a pantallas de celular.

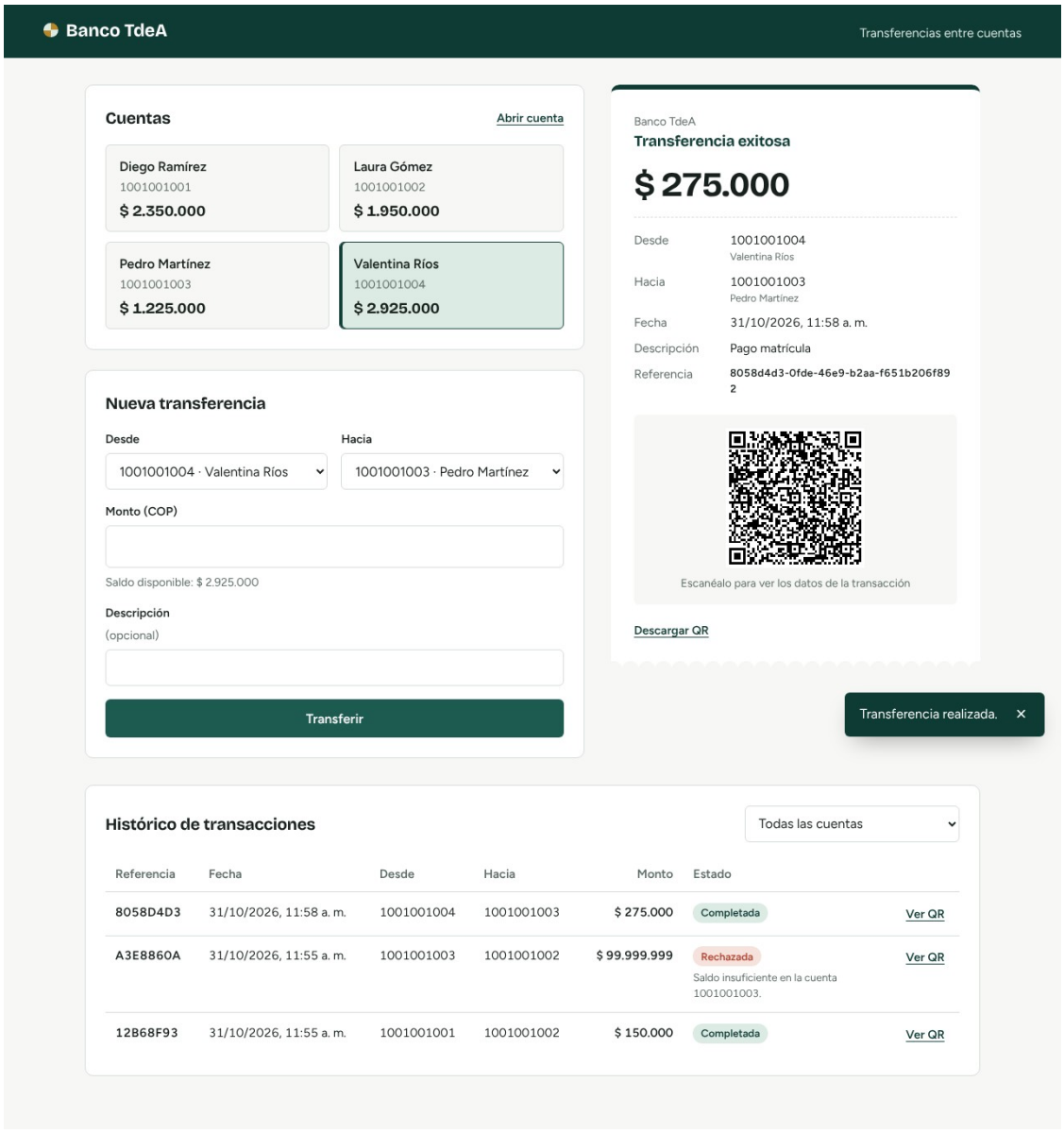


Figura 2. Transferencia exitosa con su comprobante y código QR

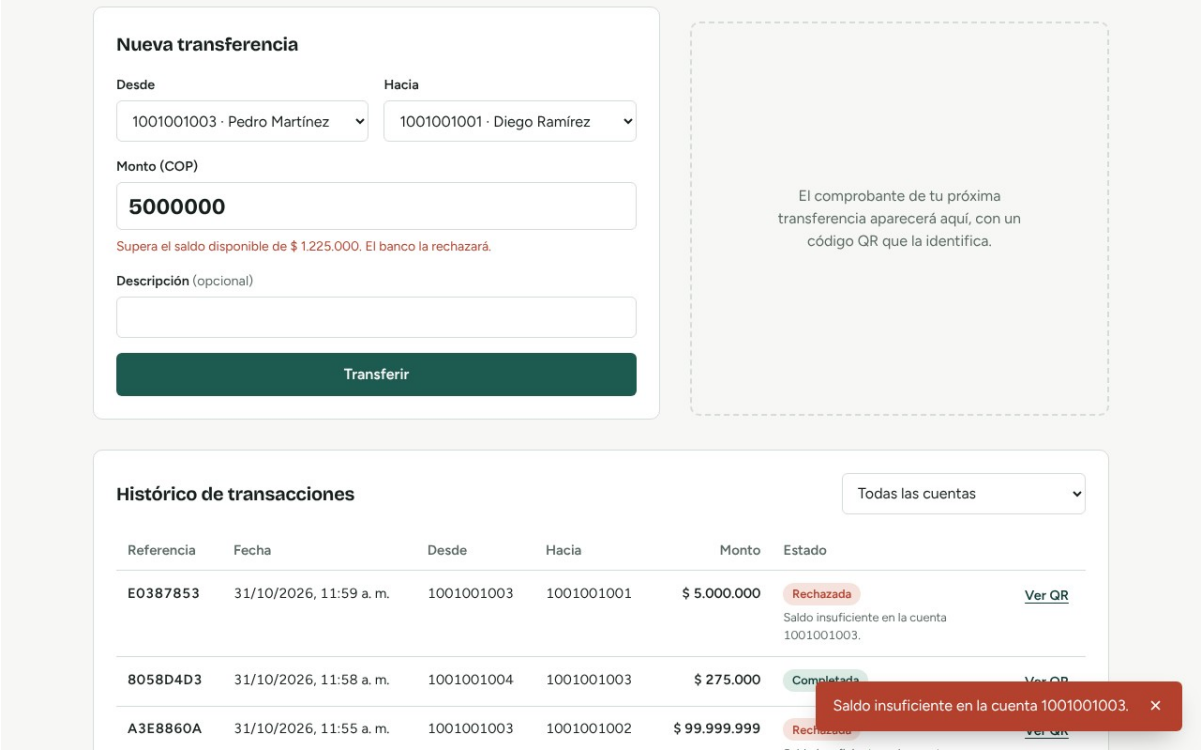


Figura 3. Transferencia rechazada por saldo insuficiente: toast de error y registro en el histórico

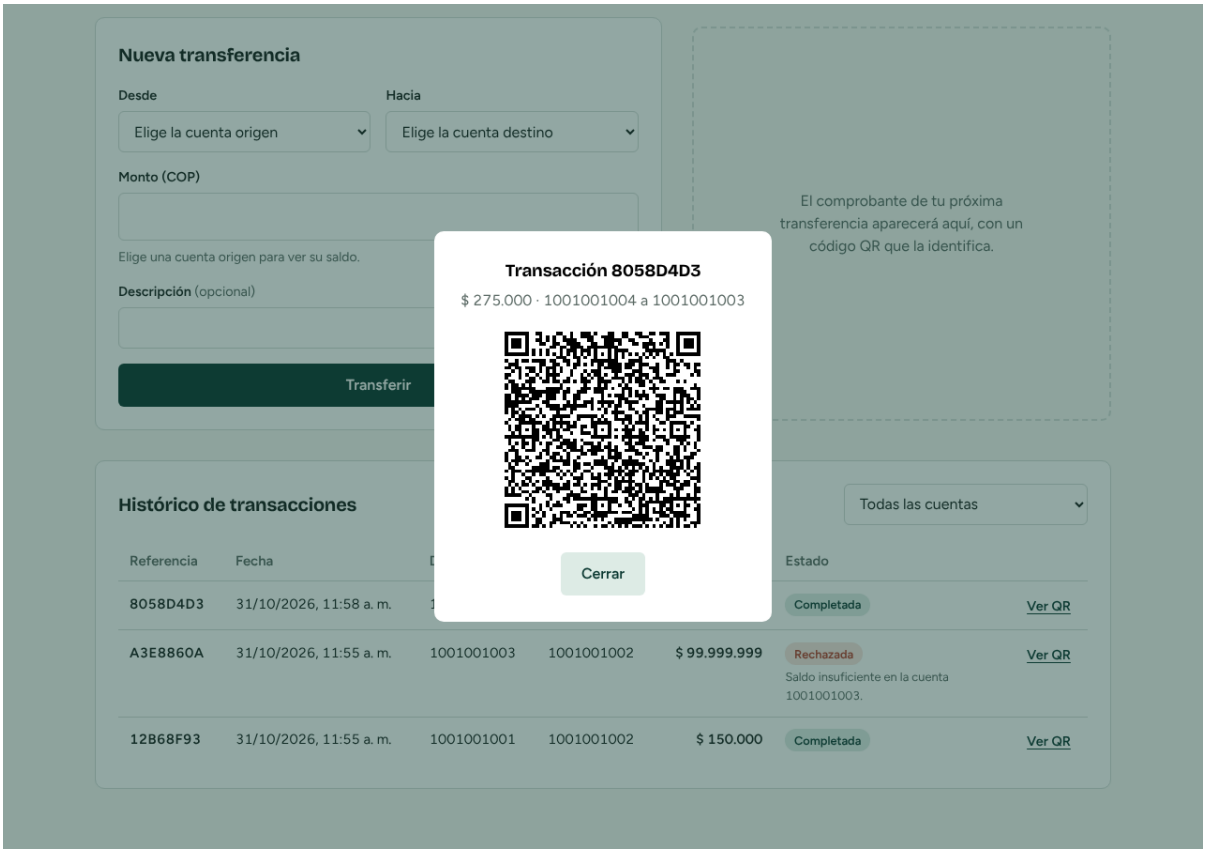


Figura 4. Código QR de una transacción consultado desde el histórico

9. Pruebas y evidencias

Pruebas automáticas

Se implementaron pruebas de integración con @SpringBootTest sobre H2. Resultado: 3 pruebas ejecutadas, 0 fallos.

- Transferencia exitosa: el dinero se mueve correctamente y la respuesta incluye el QR con la referencia.
- Saldo insuficiente: se rechaza, el saldo no cambia y el intento queda en el histórico como REJECTED.
- Misma cuenta y cuenta inexistente: se rechazan sin modificar saldos.

Pruebas manuales de la API

Caso	Código	Resultado
Transferencia válida de $150.000	201	COMPLETED + QR
Monto mayor al saldo	422	Saldo insuficiente
Campos vacíos y monto negativo	400	Mensajes de validación
Consulta del QR por referencia	200	image/png
Petición desde localhost:3000 (CORS)	200	Origen permitido

Datos guardados en PostgreSQL

```
SELECT id, source_account, target_account, amount, status FROM transactions;

 id | source_account | target_account |   amount    |  status
----+----------------+----------------+-------------+-----------
  1 | 1001001001     | 1001001002     |   150000.00 | COMPLETED
  2 | 1001001003     | 1001001002     | 99999999.00 | REJECTED
  3 | 1001001004     | 1001001003     |   275000.00 | COMPLETED
  4 | 1001001003     | 1001001001     |  5000000.00 | REJECTED
```

10. Prompts utilizados

Se siguió la misma secuencia del laboratorio. El texto completo de cada prompt está en docs/PROMPTS.md dentro del repositorio. Resumen:

#	Etapa	Qué se pidió
1	Reglas de Cursor	Archivo springboot.mdc: nombres en inglés, Javadoc, DTO como records, BigDecimal, estructura del proyecto
2	application.properties	Conexión a PostgreSQL banktdea, ddl-auto update, SQL en consola, origen CORS
3	Entidades	Account y Transaction con sus atributos y el enum TransactionStatus
4	Repositorios	Búsqueda por número, bloqueo PESSIMISTIC_WRITE, histórico

#	Etapa	Qué se pidió
		ordenado y por cuenta
5	Excepciones	Jerarquía sealed con HttpStatus y manejador global con ProblemDetail
6	DTOs	Records con validaciones y mensajes en español
7	Servicio QR	ZXing: texto de la transacción, PNG y data URI
8	Capa Service	Reglas de negocio, transacción atómica, registro de rechazos
9	Capa Controller	Endpoints de cuentas, transferencias, histórico y QR
10	Datos y pruebas	Cuentas de ejemplo y pruebas de integración con H2
11	Filtro CORS	Permitir http://localhost:3000 en todas las rutas
12	Servicio React	bankService con Axios y variable de entorno
13	Vistas React	Cuentas, formulario, comprobante con QR, histórico y toast
14	Documentación	README.md completo y subida a GitHub

11. Ejecución

- Crear la base de datos banktdea en PostgreSQL (usuario postgres, contraseña root por defecto).
- Backend: en la carpeta backend ejecutar mvn spring-boot:run. Swagger queda en http://localhost:8080/swagger-ui.html.
- Frontend: en la carpeta frontend ejecutar npm install y luego npm run dev; abrir http://localhost:3000.

12. Conclusiones

- Cursor con un archivo de reglas permanente mantiene el código consistente entre capas: nombres en inglés, Javadoc en cada método público y DTO como records, sin tener que repetirlo en cada prompt.
- En una aplicación financiera no basta con validar el saldo: la transferencia debe ser atómica y protegerse contra accesos concurrentes, lo que se resolvió con transacciones y bloqueo de filas.
- Guardar también las transferencias rechazadas hace que el histórico sea una auditoría completa y no solo una lista de movimientos exitosos.
- El código QR permite verificar los datos de una transacción sin acceder al sistema, y se puede regenerar en cualquier momento a partir de su referencia.
- La IA acelera la generación del código, pero sigue siendo necesario revisarlo y probarlo: durante el desarrollo se corrigieron detalles de diseño adaptable y del comportamiento del modal en React que solo aparecieron al probar la interfaz.